

RW BLE iOS SDK使用说明文档

1. 简介

本文档主要对SDK里所提供的功能接口与使用场景进行解释说明。

此文档仅适用于RW公司的蓝牙设备。

1.1 适用平台与语言

- iOS12及以上, 语言Objective-C.

1.2 相关术语

- App: 本文指的是手机端或平板电脑上运行的应用程序;
- 设备: 本文指的是可穿戴式硬件设备:如手表,戒指等;
- 上传: 指的是设备向App发送数据;
- 下发: 指的是App向设备发送数据;

1.3 注意事项

1. 此文档只用Objective-C语言进行说明;如果是使用swift语言开发,需要在项目的 Bridging Header 文件中导入相应的 Objective-C 头文件;
2. SDK不提供模拟器版本, 因为在模拟器上无法调试BLE且我们依赖的第三方库部分也不支持模拟器环境下运行。

2. 快速开始 (Quick Start)

第1步: 获取最新版本的Xcode

要想使用 RW BLE SDK for iOS 开发项目, 您需要安装Xcode.

第2步: 手动部署添加依赖库

手动添加 DHBLESDK.framework 到工程;

Frameworks, Libraries, and Embedded Content		
Name	Embed	
 DHBLESDK.framework	Do Not Embed	⬆️⬆️
 DHFoundation.framework	Embed & Sign	⬆️⬆️
 DHUIKit.framework	Embed & Sign	⬆️⬆️

第3步: 配置 info.plist文件

```
//需要在info.plist中对使用蓝牙进行说明
NSBluetoothAlwaysUsageDescription
NSBluetoothPeripheralUsageDescription
```

第4步: 初始化SDK

```
//AppDelegate里初始化DHBLESDK库
- (void)initBleSDK{
    [DHBLECentralManager setLogStatus:YES];
    [DHBLECentralManager initWithServiceUids:@[]];

    //Demo里工具类初始化,可选择;
    [DHBluetoothManager sharedInstance];
}
```

⚠ Caution

日志开启后 `[DHBLECentralManager setLogStatus:YES]`; 日志文件将保存在 Document/DeviceLog文件夹下。

3. 接口说明 (API Reference)

3.1 设备搜索与连接, 绑定与重连

3.1.1 搜索蓝牙

接口说明: 搜索蓝牙设备需使用 `[DHBLECentralManager startScan]`; 并实现 `DHBLEConnectDelegate` 回调接口;

返回的 `DHPeripheralModel` 实体如果 `model.macAddr` 为空, 代表手机设置是已配对设备

```
// 1. 开始搜索
[DHBLECentralManager startScan];

//2. 设备委托
[DHBLECentralManager sharedInstance].connectDelegate = self;

//3. DHBLEConnectDelegate接口会回调搜索到的蓝牙设备
- (void)centralManagerDidDiscoverPeripheral:(NSArray <DHPeripheralModel *>*)peripherals
```

3.1.2 停止搜索

接口说明: 停止搜索蓝牙设备

```
[DHBLECentralManager stopScan];
```

3.1.3 连接设备

接口说明: 连接指定设备, 并设置设备连接回调 `DHBLEConnectDelegate`.

```
// 1. 初始化并注册回调
[DHBLECentralManager sharedInstance].connectDelegate = self;

// 2. 连接设备
DHPeripheralModel *deviceModel = self.deviceArray[indexPath.row];
[DHBLECentralManager connectDeviceWithModel:deviceModel];

// 3. 实现并回调蓝牙连接状态
@protocol DHBLEConnectDelegate <NSObject>
```

`DHBLEConnectDelegate` 接口说明:

方法	说明
centralManagerDidDiscoverPeripheral	startScan 调用后,搜索到设备后会回调返回
centralManagerDidConnectPeripheral	connectDeviceWithModel后,连接成功会返回.
centralManagerDidFunctionMenu	成功获取设备配置表后会返回;业务操作应该在此之后操作.
centralManagerDidDisconnectPeripheral	蓝牙断开会回调
centralManagerDidFailedPeripheral	蓝牙失败会回调
centralManagerDidUpdateState	蓝牙开关状态变化会回调

 Tip

连接后, 业务操作应该在 `centralManagerDidFunctionMenu` 之后才进行操作.

(1) 监听 `BluetoothNotificationConnectStateChange` 也可获取连接状态变化

```
[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(connectStateChange:)
name:BluetoothNotificationConnectStateChange object:nil];

- (void)connectStateChange:(NSNotification *)ntf
{
    NSLog(@"NewHomeController connectStateChange");
    if ([DHBluetoothManager sharedInstance].isConnected){
        self.infoDeviceStateLb.text = @"Connected";
    }
    else{
        self.infoDeviceStateLb.text = @"Disconnected";
    }
}
```

(2) `[DHBluetoothManager isConnected]` 可以获取是否连接成功.

3.1.4 断开连接设备

接口说明: 断开正在连接的设备;

```
[DHBluetoothManager disconnectDevice];
```

3.1.5 绑定与自动重连,解绑

3.1.5.1 DHBluetoothManager 本地保存当前设备

设置后将保存当前设备的UDID,将会自动重新连接.

方法说明:

```
+(void)setBindedStatus:(BOOL)isBinded
```

调用示例:

```
//保存本地, 重新打开将重连
[DHBluetoothManager setBindedStatus:YES];
```

3.1.5.2 DHBleCentralManager 删除本地保存

设置后将删除当前设备的UDID, 类似于解除本地绑定.

方法说明:

```
+(void)setBindedStatus:(BOOL)isBinded
```

调用示例:

```
[DHBleCentralManager setBindedStatus:NO];
[DHBleCentralManager disconnectDevice];
```

3.1.6 设备配置表

Important

因设备型号多, 支持的功能不同,所以引入功能表信息,可查询设备功能支持情况. 具体参考DeviceFuncV2Model类. 可根据业务自行保存功能表内容.

```
-(void)centralManagerDidFunctionMenu:(DeviceFuncV2Model *)deviceFuncModel
```

DeviceFuncV2Model类属性定义:

属性	说明
isPushMsgEnableSwitch	是否启用消息控制开关
pushMsgSwitchValue	消息类型支持能力低32位 (bit0-bit31)
pushMsgSwitchValue2	消息类型支持能力高32位 (bit32-bit63) , 旧设备默认为0
activityDataInterval	当天计步明细间隔 (分钟) ; 未配置时按60处理
isAlarm	是否支持闹钟
isBackLight	是否支持屏幕睡眠时间设置
isSupportWorkout3	是否支持多运动;
isSupportMuslimCountSwitch	是否支持Muslim赞念开关
isSupportHrSpO2Alert	是否支持HR,SP02报警提示功能
isSupportMotoVibrationLevel	是否支持马达震动提醒
isSupportAlarmVibrationDuration	是否支持闹钟震动时长设置
isSupportVibrationInterval	是否支持震动间隔时长设置
isDataTypeActivity	是否支持计步
isDataTypeHeart	是否支持心率
isDataTypeBloodPressure	是否支持血压
isDataTypeSleep	是否支持睡眠
isDataTypeSPO2	是否支持血氧
isDataTypeHRV	是否支持心率变异性

isDataTypeStress	是否支持压力
isDataTypeBloodSugar	是否支持血糖
isDataTypeMuslimCount	是否支持赞念
isDataTypeTemperature	是否支持体温
isSupportMuslimTimeDisplayMode	是否支持Muslim时间显示模式
isSupportSensorRawPPG	是否支持获取PPG Green原始数据
isSupportPPGMonitoring	是否支持PPG定时监测
isSupportTemperatureMonitoring	是否支持温度定时监测
isSupportCountReminder	是否支持计数提醒间隔设置
isSupportSensorRawACC	是否支持获取ACC原始数据
isSupportSensorRawPPGRed	是否支持获取PPG Red原始数据
isSupportSensorRawIR	是否支持获取IR红外原始数据
isSupportSensorRawSleep	是否支持睡眠实时数据
isSupportFallDetect	是否支持跌落提醒
isSupportRecording	是否支持录音功能

3.1.8 使用外部 CBCentralManager 搜索并由 SDK 连接

接口说明: 当客户 App 同时集成多个 BLE SDK, 并使用自己的统一蓝牙搜索入口时, 可将搜索使用的 `CBCentralManager` 传给本 SDK。客户只负责搜索, 不负责连接; 连接、服务发现、特征发现、数据收发和断连仍由 SDK 处理。

Important

`CBPeripheral` 与扫描它的 `CBCentralManager` 是绑定的。客户传入的 `peripheral` 必须来自当前设置的 `externalCentralManager`。SDK 不使用私有 API 校验来源, 如果传入的 Central 与 Peripheral 不匹配, 连接可能失败或超时。

方法说明:

```
// 初始化 SDK, 并指定外部 CBCentralManager。centralManager 传 nil 时使用 SDK 自管理模式。
+ (void)initWithServiceUuids:(NSArray <NSString *>*)uuids
    externalCentralManager:(nullable CBCentralManager *)centralManager;

// 运行中设置外部 CBCentralManager。只能在未连接、未连接中状态下设置。
+ (void)setExternalCentralManager:(nullable CBCentralManager *)centralManager;

// 是否正在使用外部 CBCentralManager。
+ (BOOL)isUsingExternalCentralManager;

// 连接客户统一搜索到的设备。
+ (void)connectPeripheral:(CBPeripheral *)peripheral
    advertisementData:(nullable NSDictionary *)advertisementData
    RSSI:(nullable NSNumber *)RSSI;
```

参数说明:

参数	类型	说明
centralManager	CBCentralManager	客户统一搜索使用的同一个 Central。SDK 后续会使用它发起连接。
peripheral	CBPeripheral	客户通过 <code>centralManager</code> 搜索到的设备对象，连接必须使用该对象。
advertisementData	NSDictionary	广播数据，用于补全 <code>DHPeripheralModel</code> 的 <code>macAddr</code> 、 <code>deviceModel</code> 、设备名等信息；连接本身不依赖该参数，可传 <code>nil</code> 。
RSSI	NSNumber	信号强度，用于补全 <code>DHPeripheralModel.rssi</code> ；连接本身不依赖该参数，可传 <code>nil</code> 。

调用示例:

```
// 1. 客户使用自己的 CBCentralManager 统一搜索设备。
// centralManager、peripheral、advertisementData、RSSI 来自客户自己的扫描回调。

// 2. 将同一个 CBCentralManager 交给 SDK。
[DHBLECentralManager initWithServiceUids:@[]
    externalCentralManager:centralManager];

// 或者 SDK 已初始化时，运行中设置外部 Central。
[DHBLECentralManager setExternalCentralManager:centralManager];

// 3. 设置 SDK 连接回调。
[DHBLECentralManager sharedInstance].connectDelegate = self;

// 4. 由 SDK 发起连接。
[DHBLECentralManager connectPeripheral:peripheral
    advertisementData:advertisementData
    RSSI:RSSI];
```

注意事项:

- 旧接入方式 `[DHBLECentralManager initWithServiceUids:@[]]` 保持不变，SDK 会自行创建 `CBCentralManager`。
- 外部 Central 模式下，SDK 会使用客户传入的同一个 `CBCentralManager` 连接设备。
- SDK 开始连接后会接管 `centralManager.delegate`，后续连接回调由 SDK 处理。
- 客户应在交给 SDK 连接前停止自己的扫描流程。
- 连接中、已连接、正在服务发现或正在同步数据时，不允许切换 `externalCentralManager`。
- 外部 Central 模式下如需自动重连，客户需要保持传入的 `centralManager` 实例存活。

3.2 设备功能操作

调用接口发送错误码定义 SendStateCode:

SendStateCode	值	说明
SendState_OK	0	成功
SendState_BLENoOpen	1	失败, 蓝牙未打开
SendState_BLEDisconnect	2	失败, 蓝牙未连接
SendState_TimeOut	3	失败, 超时
SendState_Failed	4	失败
SendState_DuplicateSend	5	有重复指令
SendState_NotSupport	6	设备不支持

3.2.1 基础功能指令接口

3.2.1.0 Get SDK Version

获取SDK版本号.

方法说明:

[DHBLECommand getSDKVersion]

调用示例:

```
NSLog(@"%@", [DHBLECommand getSDKVersion]);
```

3.2.1.1 获取蓝牙Mac地址

因iOS系统配对后,广播包无法拿到mac地址,提供此方法可获取.

方法说明:

+ (void)ringGetMacAddress:(void(^)(int code, id data))block

返回参数说明:

参数	类型	说明	值
model	DHDeviceInfoModel	类	macAddr: Mac地址

调用示例:

```
[DHBLECommand ringGetMacAddress:^(int code, id _Nonnull data) {
    DHDeviceInfoModel *tDeviceInfoData = data;
    NSLog(@"mac: %@", tDeviceInfoData.macAddr);
}];
```

3.2.1.2 设置用户信息

用户信息设置与计步卡路里,距离有关; 设备初始化时性别为1, 年龄 18, 身高170cm, 体重 65 kg.

方法说明:

```
+ (void)setUserInfo:(DHUserInfoSetModel *)model block:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
model	DHUserInfoSetModel	类	gender: 性别 (0.女 1.男) height: 身高cm,浮点型 weight: 体重kg,浮点型 stepGoal: 计步目标值,暂无功能 age: 年龄

调用示例:

```
DHUserInfoSetModel *userInfoModel = [[DHUserInfoSetModel alloc] init];
userInfoModel.gender = 1;
userInfoModel.height = 170;
userInfoModel.weight = 600;
userInfoModel.stepGoal = 8000;
userInfoModel.age = 20;
[DHBleCommand setUserInfo:userInfoModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"set ok");
    }
}
}];
```

3.2.1.3 获取固件信息

接口说明: 获取固件型号,固件版本号,UI版本号;

```
[DHBleCommand getFirmwareVersion:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"getFirmwareVersion OK");
        DHFirmwareVersionModel *model = data;
        NSLog(@"型号 %@ 固件版本 %@ UI版本 %@", model.deviceModel, model.firmwareVersion,
model.uiVersion);
    }
}
}];
```

3.2.1.4 获取电量

接口说明: app获取设备电量

```
[DHBleCommand getBattery:^(int code, id _Nonnull data) {
    if (code == 0){
        DHBatteryInfoModel *model = data;
        NSLog(@"getBattery OK 电量值 %zd", model.battery);
    }
}
}];
```

3.2.1.5 获取与设置视频控制开关

设置是否开启戒指手势控制刷视频; 此功能需配对蓝牙HID.

```
//设置视频控制开关
DHVideoHidSetModel *tModeSetModel = [[DHVideoHidSetModel alloc] init];
tModeSetModel.isOpen = YES;
[DHBleCommand setVideoHid:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setVideoHid OK");
    }
}]];

// 获取视频控制开关
[DHBleCommand getVideoHid:^(int code, id _Nonnull data) {
    if (code == 0){
        DHVideoHidSetModel *model = data;
        NSLog(@"getVideoHid OK 开关 %d", model.isOpen);
    }
}]];
```

3.2.1.6 获取与设置LED亮屏强度

方法说明:

```
+(void)getRingLEDLight:(void(^)(int code, id data))block;

+(void)setRingLEDLight:(DHLedLightSetModel *)model block:(void(^)(int code, id data))block;
```

参数说明:

参数	类型	说明	值
model	DHLedLightSetModel	类	isOpen: false为off,ture为(1-3Level) lightLevel: 1微光, 2柔光, 3强光

调用示例:

```
// 获取LED亮屏强度
[DHBleCommand getRingLEDLight:^(int code, id _Nonnull data) {
    if (code == 0){
        DHLedLightSetModel *model = data;
        NSLog(@"getRingLEDLight OK 开关 %d", model.isOpen);
    }
}]];

//设置LED亮屏强度
DHLedLightSetModel *tModeSetModel = [[DHLedLightSetModel alloc] init];
tModeSetModel.isOpen = YES; //设置为NO关闭
tModeSetModel.lightLevel = 3; //1微光2柔光3强光
[DHBleCommand setRingLEDLight:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setRingLEDLight OK");
    }
}]];
```

3.2.1.7 获取与设置佩戴位置

```
// 获取佩戴位置
[DHBluetoothCommand getRingWearHand:^(int code, id _Nonnull data) {
    if (code == 0){
        NSInteger tWearHand = [data intValue]; //0左手 1右手
        NSLog(@"getRingWearHand OK 佩戴位置 %zd", tWearHand);
    }
}];

//设置佩戴位置
uint8_t tModeSetModel = 0; //0左手 1右手
[DHBluetoothCommand setRingWearHand:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setRingWearHand OK");
    }
}];
```

3.2.1.8 启动与关闭拍照

启动拍照功能后,设备可通过手势控制app自定义相机拍照.

配置表属性: `isTakePhoto`

`BluetoothNotificationCameraTakePicture` 设备发出拍照通知,进行拍照

```
//APP进相机界面启动 1为控制设备进对应界面, 0为控制设备退出
[DHBluetoothCommand controlCamera:1 block:^(int code, id _Nonnull data) {

}];

//监听设备发出拍照指令
[[NSNotificationCenter defaultCenter] addObserver:self
selector:@selector(cameraTakePictureNotification) name:BluetoothNotificationCameraTakePicture
object:nil];

- (void)cameraTakePictureNotification {
    //进行拍照
    NSLog(@"cameraTakePictureNotification 进行拍照");
}
```

3.2.1.9 查找设备

调用查找后设备灯或屏幕会亮.

方法说明:

```
+(void)controlFindDeviceBegin:(void(^)(int code, id data))block;
```

调用示例:

```
[DHBluetoothCommand controlFindDeviceBegin:^(int code, id _Nonnull data) {

}];
```

3.2.1.10 关机,恢复出厂设置

方法说明:

```
+(void)controlDevice:(NSInteger)type block:(void(^)(int code, id data))block;
```

参数说明:

参数	类型	说明	值
type	NSInteger	整形	1: 关机 2: 恢复出厂

调用示例:

```
// 1关机 2 恢复出厂
[DHBleCommand controlDevice:1 block:^(int code, id _Nonnull data) {

}];
```

3.2.1.11 闹钟

配置表属性: isAlarm

3.2.1.11.1 获取已设置闹钟

方法说明:

```
+(void)getAlarms:(void(^)(int code, id data))block
```

调用示例:

```
//获取设备里已保存的闹钟
[DHBleCommand getAlarms:^(int code, id _Nonnull data) {
    if (code == 0) {
        NSArray *tAlarmList = data;
        NSLog(@"getAlarms %zd", tAlarmList.count);
    }
}];
```

3.2.1.11.2 设置闹钟

当前协议不支持单独修改闹钟，任何单个闹钟的开关或删除操作，均需重新下发全部闹钟配置。

方法说明:

```
+(void)setAlarms:(NSArray <DHAlarmSetModel *>*)alarms block:(void(^)(int code, id data))block;
```

参数说明:

参数	类型	说明	值
alarms	List	闹钟数组	isOpen: true开/false关 repeats: IntArray(7)周日至周六,要重复的对应置1 hour: 闹钟开始时 minute: 闹钟开始分

调用示例:

```
//设置闹钟
DHAlarmSetModel *tAlarm1 = [[DHAlarmSetModel alloc] init];
```

```

tAlarm1.hour = 07; //时
tAlarm1.minute = 00; //分
tAlarm1.isOpen = true; //开关
tAlarm1.repeats = @[0,0,0,0,0,0,1]; //重复周期,周六重复

DHAlarmSetModel *tAlarm2 = [[DHAlarmSetModel alloc] init];
tAlarm2.hour = 8;
tAlarm2.minute = 00;
tAlarm2.isOpen = true;
tAlarm2.repeats = []; //单次闹钟

[DHBleCommand setAlarms:@[tAlarm1, tAlarm2] block:^(int code, id _Nonnull data) {

}
}];

```

3.2.1.11.3 删除所有闹钟

setAlarms 参数传空数组,即可删除所有闹钟

```

[DHBleCommand setAlarms:@[] block:^(int code, id _Nonnull data) {

}
}];

```

3.2.1.12 震动次数设置与获取

设置设备震动次数;

配置表属性: isSupportMotoVibrationLevel

方法说明:

```

+(void)setRingMotorLevel:(NSInteger)motorLevel motorNum:(NSInteger)motorNum block:(void(^)(int code, id data))block;

```

```

+(void)getRingMotorLevel:(void(^)(int code, id data))block;

```

参数说明:

参数	类型	说明	值
motorLevel	Int	整形	震动强度, 0:关闭 1:低 2: 中 3: 高; 未定义此功能的可忽略
motorNum	Int	整形	震动次数可以被设置(0-6次), 初始默认2次.设置0次不震动

调用示例:

```

//设置
[DHBleCommand setRingMotorLevel:1 motorNum:2 block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"set ok");
    }
}
}];

//获取
[DHBleCommand getRingMotorLevel:^(int code, id _Nonnull data) {
    if (code == 0){
        DHVibrationLevelModel *tVibrationModel = data;
    }
}
}];

```

```
        NSLog(@"getRingMotorLevel Level %d num %d", tVibrationModel.vibrationLevel,
tVibrationModel.vibrationNumber);
    }
}];
```

3.2.1.13 屏幕睡眠模式设置与获取

- 设置屏幕睡眠开启与时间;
- 配置表属性: isBackLightSleepMode

方法说明:

```
+(void)setDisplaySleepMode:(DHBrightTimeSetModel *)model block:(void(^)(int code, id data))block
+(void)getDisplaySleepMode:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
DHBrightTimeSetModel	类		sleepOpen, 开关打开YES或关闭NO sleepStartHour, 开始时间小时 sleepStartMin,开始时间分钟 sleepEndHour,结束时间小时 sleepEndMin,结束时间分钟

调用示例:

```
//设置
DHBrightTimeSetModel *sleepModel = [[DHBrightTimeSetModel alloc] init];
sleepModel.sleepOpen = YES;
sleepModel.sleepStartHour = 20;
sleepModel.sleepStartMin = 00;
sleepModel.sleepEndHour = 06;
sleepModel.sleepEndMin = 00;
[DHBleCommand setDisplaySleepMode:sleepModel block:^(int code, id _Nonnull data) {

}];

//获取
[DHBleCommand getDisplaySleepMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHBrightTimeSetModel *tModel = data;
        NSLog(@"getDisplaySleepMode sleepOpen %d sleepStartHour %d sleepStartMin %d", tModel.sleepOpen,
tModel.sleepStartHour, tModel.sleepEndMin);
    }
}];
```

3.2.1.14 消息推送开关设置与获取

配置表属性 `isPushMsgEnableSwitch` 表示设备是否支持消息推送开关设置；
`pushMsgSwitchValue` 和 `pushMsgSwitchValue2` 是消息类型支持能力，分别对应 bit0-bit31 和 bit32-bit63，不表示设备当前的开关状态。

进入消息推送设置页面时，应调用 `ringGetAncs:` 从设备拉取当前开关。
查询成功后，`data` 为 `DHAncsSetModel`；修改模型后再调用 `ringSetAncs:block:` 写回设备。

用途	属性/接口	返回内容
判断功能是否支持	<code>DeviceFuncV2Model.isPushMsgEnableSwitch</code>	是否支持消息推送开关
判断消息类型是否支持	<code>pushMsgSwitchValue</code> / <code>pushMsgSwitchValue2</code>	支持的消息类型 bit0-bit63
拉取当前开关	<code>ringGetAncs:</code>	<code>DHAncsSetModel</code>
设置当前开关	<code>ringSetAncs:block:</code>	设置结果

方法说明:

```
+(void)ringSetAncs:(DHAncsSetModel *)model block:(void(^)(int code, id data))block

+(void)ringGetAncs:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
DHAncsSetModel	类		见DHAncsSetModel类定义

调用示例:

```
//设置
DHAncsSetModel *tAncsModel = [[DHAncsSetModel alloc] init];
tAncsModel.isSMS = YES;
[DHBleCommand ringSetAncs:tAncsModel block:^(int code, id _Nonnull data) {

}];

//获取
[DHBleCommand ringGetAncs:^(int code, id _Nonnull data) {
    if (code == 0){
        DHAncsSetModel *ancsModel = data;
        NSLog(@"SMS=%d Wechat=%d", ancsModel.isSMS, ancsModel.isWechat);
    }
}];

//按下面的顺序确定当前设备支持哪些应用消息。
UInt8 tBitRow = i;
if (i == 24){ //其它
    tBitRow = 0;
}
if (i > 0 && (tMsgSwitchValue & (1 << tBitRow)) < 1){ //不支持的消息全设置为false,不管从设备读到的值。
    continue;
}

self.msgOpenFlagArr = [NSMutableArray arrayWithArray:@[
    @(self.model.isCall),
```

```

        @(self.model.isSMS),
        @(self.model.isEmail),
        @(self.model.isSkype),
        @(self.model.isFacebook),
        @(self.model.isWhatsapp),
        @(self.model.isLine),
        @(self.model.isInstagram),
        @(self.model.isKakaotalk),
        @(self.model.isGmail),
        @(self.model.isTwitter),
        @(self.model.isLinkedin),
        @(self.model.isJLSinaWeiBo),

        @(self.model.isQQ),
        @(self.model.isWechat),
        @(self.model.isJLBand),
        @(self.model.isJLTelegram),
        @(self.model.isJLBetween),
        @(self.model.isJLNavercafe),
        @(self.model.isYoutube),
        @(self.model.isJLNetflix), //(1<<21)
        @(self.model.isMax), // (1<<22)
        @(self.model.isVkim), // (1<<23)
        //          @(self.model.isMessenger),

        @(self.model.isOther)
    ];
}

```

3.2.1.15 获取与设置赞念是否打开

设置赞念功能是否打开;

配置表属性: `isSupportMuslimCountSwitch`

方法说明:

```
+(void)setMuslimCountSwitch:(UInt8)isOpen block:(void(^)(int code, id data))block
```

```
+(void)getMuslimCountSwitch:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
isOpen	Int		0: 关闭 1: 打开

调用示例:

```

//设置
[DHBluetooth setMuslimCountSwitch:1 block:^(int code, id _Nonnull data) {

}];

//获取
[DHBluetooth getMuslimCountSwitch:^(int code, id _Nonnull data) {
    if (code == 0){

```

```

        Boolean tOpen = [data boolValue];
        NSLog(@"getMuslimCountSwitch tOpen %d", tOpen);
    }
}];

```

3.2.1.16 获取与设置心率/血氧报警配置

设置心率与血氧通知报警数据功能; 报警提示会通 `BluetoothNotificationRingHealthOverAlert` 通知出来.

配置表属性: `isSupportHrSpO2Alert`

方法说明:

```
+(void)getHRAAlert:(void(^)(int code, id data))block
```

```
+(void)setHRAAlert:(DHHRAlertModel *)overModel block:(void(^)(int code, id data))block
```

```
+(void)getSP02Alert:(void(^)(int code, id data))block
```

```
+(void)setSP02Alert:(DHHRAlertModel *)overModel block:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
DHHRAlertModel	类		isOpen: YES 开, NO 关; overValue: 报警值, 默认值为 心率超过160, 血氧低于94%; underValue: 低于设置值报警; 如果获取到为0xff,代表不支持此项功能;

注意: 通过 `getHRAAlert()`获取如果underValue为0xff,代表不支持此项功能.

调用示例:

```

//设置
DHHRAlertModel *tHRAAlertModel = [[DHHRAlertModel alloc] init];
tHRAAlertModel.isOpen = YES;
tHRAAlertModel.overValue = 160;
tHRAAlertModel.underValue = 0xff;
[DHBleCommand setHRAAlert:tHRAAlertModel block:^(int code, id _Nonnull data) {

}];

//获取
[DHBleCommand getHRAAlert:^(int code, id _Nonnull data) {
    if (code == 0){
        DHHRAlertModel *model = data;
        NSLog(@"getHRAAlert %d %zd", model.isOpen, model.overValue);
    }
}];

//报警结果通知推送
[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(healthOverAlert:)
name:BluetoothNotificationRingHealthOverAlert object:nil];

```



```

- (void)healthOverAlert:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    NSInteger tType = [tUserInfo[@"type"] integerValue];
    NSInteger tValue = [tUserInfo[@"value"] integerValue];

    if (tType == 0){ //HeartRate Alert
    }
    else if (tType == 1){ //SP02
    }
    else if (tType == 2){ //HeartRate Alert Under
    }
}
}

```

3.2.1.17 获取与设置亮屏时长

配置表属性: `isBackLight`

方法说明:

```
+(void)getBrightTime:(void(^)(int code, id data))block
```

```
+(void)setBrightTime:(DHBrightTimeSetModel *)model block:(void(^)(int code, id data))block
```

参数说明:

DHBrightTimeSetModel参数	类型	说明	值
duration	Int	亮屏时长,秒(s), 范围0-30s;	
durationNums	String	设备支持的时长值, 获取到如果有值,以逗号隔开;	

3.2.1.18 获取与设置抬手亮屏时长

配置表属性: `isSupportRaisescreen`

方法说明:

```
+(void)ringGetGesture:(void(^)(int code, id data))block;
```

```
+(void)ringSetGesture:(DHGestureSetModel *)model block:(void(^)(int code, id data))block;
```

参数说明:

BrightScreenBean参数	类型	说明	值
isOpen	Int	true:开启; false: 关闭	
startHour	Int	开始时间小时	
startMin	Int	开始时间分钟	
endHour	Int	结束时间小时	
endMin	Int	结束时间分钟	

3.2.1.19 设置时间格式12/24小时制

带屏显示时间的设备才有效.

方法说明:

```
+(void)ringSetTimeformat:(UInt8)timeformat block:(void(^)(int code, id data))block;
```

参数说明:

参数	类型	说明	
timeformat	Int	0: 24小时制 1: 12小时制	

3.2.1.20 闹钟震动时长设置与获取

设置闹钟震动次数;

配置表属性: isSupportAlarmVibrationDuration

方法说明:

```
+(void)setAlarmVibrationDuration:(UInt8)count block:(void(^)(int code, id data))block
```

```
+(void)getAlarmVibrationDuration:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
count	UInt8	整形	震动次数(0-6), 默认2次, 设置0次为不震动

调用示例:

```
//设置
[DHBleCommand setAlarmVibrationDuration:2 block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setAlarmVibrationDuration OK");
    }
}];

//获取
[DHBleCommand getAlarmVibrationDuration:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"getAlarmVibrationDuration count %@", data);
    }
}];
```

3.2.1.21 触摸事件通知

设备触摸事件通知, 设备主动上报. 触摸操作无论熄屏与否都会上报, 由APP定义响应行为.

通过 `BluetoothNotificationTouchEvent` 通知返回.

通知 userInfo 数据说明:

字段	说明	值
keyType	按键类型	1: 触摸按键(默认), 2: 跌落(需开启跌落提醒3.2.1.24)
touchType	触摸类型	1: 单击, 2: 双击, 3: 三击, 4: 长按, 5: 甩动. 按键类型为2(跌落)时, 触摸类型默认为1

调用示例:

```
[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(touchEventNotification:)
name:BluetoothNotificationTouchEvent object:nil];

- (void)touchEventNotification:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    NSInteger keyType = [tUserInfo[@"keyType"] integerValue]; // 1:触摸按键
    NSInteger touchType = [tUserInfo[@"touchType"] integerValue]; // 1:单击 2:双击 3:三击 4:长按 5:甩动
    NSLog(@"TouchEvent keyType=%zd touchType=%zd", keyType, touchType);
}
```

3.2.1.22 震动间隔时长设置与获取

设置每次震动之间的间隔时间, 用于调节震动节奏;

配置表属性: `isSupportVibrationInterval`

方法说明:

```
+(void)setVibrationInterval:(UInt16)intervalMs block:(void(^)(int code, id data))block
```

```
+(void)getVibrationInterval:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
intervalMs	UInt16	整形	间隔时长(100-1000ms), 默认500ms

调用示例:

```
//设置
[DHBleCommand setVibrationInterval:500 block:^(int code, id _Nonnull data) {
    if (code == 0){ NSLog(@"setVibrationInterval OK"); }
}];

//获取
[DHBleCommand getVibrationInterval:^(int code, id _Nonnull data) {
    if (code == 0){ NSLog(@"getVibrationInterval %@ms", data); }
}];
```

3.2.1.23 心率校正(工厂测试)

启动设备心率校正模式. 发送校正指令后, 设备会返回两条数据:
第1条 result=0 表示校正中; 第2条 result非0 表示校正完成.
block 回调 data 为 NSDictionary: testMode(UInt8) + result(UInt32, 0=校正中, 非0=完成).

方法说明:

```
+(void)startFactoryTest:(UInt8)testMode block:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
testMode	UInt8	测试模式	0x15: 心率校正

调用示例:

```
[DHBleCommand startFactoryTest:0x15 block:^(int code, id _Nonnull data) {
    if (code == 0 && [data isKindOfClass:[NSDictionary class]]){
        NSDictionary *info = data;
        NSInteger result = [info[@"result"] integerValue];
        if (result == 0){
            NSLog(@"HR calibrating...");
        } else {
            NSLog(@"HR calibration done, result=%zd", result);
        }
    }
}];
```

3.2.1.24 跌落提醒设置

设置或获取跌落提醒开关. 开启后设备检测到跌落时会通过触摸事件通知(3.2.1.21)上报.
跌落事件通过 BluetoothNotificationRingTouchEvent 通知返回, 按键类型(keyType)=2 表示跌落事件.
配置表属性: isSupportFallDetect

方法说明:

```
+(void)setFallDetect:(UInt8)enable block:(void(^)(int code, id data))block

+(void)getFallDetect:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
enable	UInt8	开关	0: 关, 1: 开

调用示例:

```
//获取跌落提醒开关
[DHBLECommand getFallDetect:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"getFallDetect OK: %@", data);
    }
}];

//设置跌落提醒开启
[DHBLECommand setFallDetect:1 block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setFallDetect ON OK");
    }
}];
```

3.2.1.25 计数提醒间隔设置

设置或获取计数提醒间隔. 开启后用户完成一次计数操作开始计时, 达到设定间隔后设备震动一次提醒继续计数.

配置表属性: isSupportCountReminder

方法说明:

```
+(void)setCountReminderInterval:(UInt8)interval block:(void(^)(int code, id data))block

+(void)getCountReminderInterval:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
interval	UInt8	间隔分钟	0: 关闭, 30/60/90/120: 提醒间隔(分钟)

调用示例:

```
//获取计数提醒间隔
[DHBLECommand getCountReminderInterval:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"CountReminderInterval: %@ min", data);
    }
}];

//设置计数提醒间隔60分钟
[DHBLECommand setCountReminderInterval:60 block:^(int code, id _Nonnull data) {
```

```
        if (code == 0){
            NSLog(@"setCountReminderInterval OK");
        }
    }
}

//关闭计数提醒
[DHBleCommand setCountReminderInterval:0 block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"CountReminder OFF");
    }
}
}];
```

3.2.2 健康数据同步(实时单次与全天检测)

健康数据检测有两种方式: 实时单次检测与全天检测。健康数据包括心率,血氧,压力,HRV,睡眠等, 睡眠无实时检测。

(1) 实时单次检测: APP侧启动设备进入单次检测,检测完后马上返回结果。

(2) 全天检测: 可设置间隔时间,比如30分钟或60分钟设备会进行检测并保存值; **app一直不同步情况下,设备可保存3-6天的数据。**

3.2.2.1 实时检测-启动与关闭设备健康数据检测

启动健康数据检测(心率,血氧,HRV,压力,血糖);

测试完成设备通过 BluetoothNotificationHealthRingMeasureStateChange 通知app;

测试实时值通过 BluetoothNotificationHealthRingMeasureValueChange 通知app;

⚠ Caution

同一时间只能开启一种健康检测类型, 必须等当前检测结束(收到完成回调)或主动关闭后, 才能启动新的检测类型. 同时开启多种会导致检测异常.

方法说明:

```
+(void)controlOpen:(NSInteger)type dataType:(NSInteger)dataType block:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
dataType	NSInteger	健康数据类型	心率: BLE_KEY_HEART_RATE 血氧: BLE_KEY_BLOOD_OXYGEN HRV: BLE_KEY_HRV 压力: BLE_KEY_STRESS 血糖: BLE_KEY_BLOOD_SUGAR 血压: BLE_KEY_BLOOD_PRESSURE 体温: BLE_KEY_TEMPERATURE
type	NSInteger	启动/关闭	启动: 1 关闭: 0

调用示例:

```

+ (void)controlOpen:(NSInteger)type dataType:(NSInteger)dataType block:(void(^)(int code, id
data))block
//启动心率测试
[DHBleCommand controlOpen:1 dataType:BLE_KEY_HEART_RATE block:^(int code, id _Nonnull data) {

}];

//关闭心率测试
[DHBleCommand controlOpen:0 dataType:BLE_KEY_HEART_RATE block:^(int code, id _Nonnull data) {

}];

// 监听测量中实时数值改变
[[NSNotificationCenter defaultCenter] addObserver:self
selector:@selector(updateRingMeasureValueChange:)
name:BluetoothNotificationHealthRingMeasureValueChange object:nil];

- (void)updateRingMeasureValueChange:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    NSInteger tDataValue = [tUserInfo[@"dataValue"] integerValue];
    NSInteger tDataType = [tUserInfo[@"dataType"] integerValue];

    NSLog(@"updateRingMeasureValueChange 0x%04X 数值: %zd", (unsigned int)tDataType, tDataValue);
    if (tDataType == BLE_KEY_APP_REAL_TIME_MUSLIM_COUNT){ //Muslim Count

    }
    else if (tDataType == BLE_KEY_APP_REAL_BLOOD_SUGAR_DATA){ //BloodSugar 血糖

    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_HRV_DATA){ //HRV 心率变异性

    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_HR_DATA){ //HR 心率

    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_BLOOD_OXYGEN_DATA){ //BloodOxygen 血氧

    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_STRESS_DATA){ //Stress 压力

    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_BP_DATA){ //BloodPressure 血压
        NSInteger sp = [tUserInfo[@"systolic"] integerValue]; //收缩压
        NSInteger dp = [tUserInfo[@"diastolic"] integerValue]; //舒张压
        NSLog(@"BloodPressure sp=%zd dp=%zd", sp, dp);
    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_TEMPERATURE_DATA){ //Temperature 体温
        NSLog(@"Temperature value=%.1f", tDataValue / 10.0); //除以10得实际温度
    }
}
}

```

3.2.2.2 全天检测-设置健康数据全天监听间隔

设置健康数据(心率,血氧,HRV,压力,血糖)全天监听间隔, 单位分钟.

注意事项:暂间隔只有心率可设置30分钟与60分钟, 其它(血氧,HRV,压力,血糖)只能设置开与关; 开始与结束时间固定全天,不可修改.

3.2.2.2.1 心率检测设置与获取

间隔只有心率可设置30分钟与60分钟;

方法说明:

```
+(void)setHeartRateMode:(DHHeartRateModeSetModel *)model block:(void(^)(int code, id data))block
+(void)getHeartRateMode:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
model	DHHeartRateModeSetModel	类	isOpen: true开/false关 interval: 间隔时间30或60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 1. Set HeartRate Monitor(设置心率监听)
DHHeartRateModeSetModel *tModeSetModel = [[DHHeartRateModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 30; //可设置30分钟或60分钟
[DHbleCommand setHeartRateMode:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setHeartRateMode OK");
    }
}];

//1. 获取心率监听
[DHbleCommand getHeartRateMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHHeartRateModeSetModel *model = data;
        NSLog(@"getHeartRateMode OK 开关 %d 检测周期 %d", model.isOpen, model.interval);
    }
}];
```

3.2.2.2.2 血氧检测设置与获取

间隔血氧只可设置60分钟;

方法说明:

```
+(void)setBoMode:(DHBoModeSetModel *)model block:(void(^)(int code, id data))block
+(void)getBoMode:(void(^)(int code, id data))block
```


参数说明:

参数	类型	说明	值
model	DHBoModeSetModel	类	isOpen: true开/false关 interval: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 2. Set Blood oxygen Monitor(设置血氧监听)
DHBoModeSetModel *tModeSetModel = [[DHBoModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBleCommand setBoMode:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setBoMode OK");
    }
}];

//2. 获取血氧监听
[DHBleCommand getBoMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHBoModeSetModel *model = data;
        NSLog(@"getBoMode OK 开关 %d 检测周期 %d", model.isOpen, model.interval);
    }
}];
```

3.2.2.2.3 心率变异性(HRV)检测设置与获取

间隔HRV只可设置60分钟;

方法说明:

```
+(void)setHrvMode:(DHHrvModeSetModel *)model block:(void(^)(int code, id data))block

+(void)getHrvMode:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
model	DHHrvModeSetModel	类	isOpen: true开/false关 interval: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 3. Set HRV Monitor(设置HRV监听)
DHHRVModeSetModel *tModeSetModel = [[DHHRVModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBleCommand setHrvMode:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setHrvMode OK");
    }
}];

//3. 获取HRV监听
[DHBleCommand getHrvMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHHRVModeSetModel *model = data;
        NSLog(@"getHrvMode OK 开关 %d", model.isOpen);
    }
}];
```

3.2.2.2.4 压力检测设置与获取

间隔压力只可设置60分钟;

方法说明:

```
+(void)setStressMode:(DHStressModeSetModel *)model block:(void(^)(int code, id data))block
+(void)getStressMode:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
model	DHStressModeSetModel	类	isOpen: true开/false关 interval: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 4. 设置压力监听
DHStressModeSetModel *tModeSetModel = [[DHStressModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBleCommand setStressMode:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
```

```

        NSLog(@"setStressMode OK");
    }
}];

//4. 获取压力监听
[DHBLECommand getStressMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHStressModeSetModel *model = data;
        NSLog(@"getStressMode OK 开关 %d", model.isOpen);
    }
}];

```

3.2.2.2.5 血糖检测设置与获取

间隔血糖只可设置60分钟;

配置表属性: `isDataTypeBloodSugar`

方法说明:

```

+(void)setBloodSugarMode:(DHBloodSugarModeSetModel *)model block:(void(^)(int code, id data))block

+(void)getBloodSugarMode:(void(^)(int code, id data))block

```

参数说明:

参数	类型	说明	值
model	DHBloodSugarModeSetModel	类	isOpen: true开/false关 interval: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```

// 5. 设置血糖监听
DHBloodSugarModeSetModel *tModeSetModel = [[DHBloodSugarModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBLECommand setBloodSugarMode:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setBloodSugarMode OK");
    }
}];

//5. 获取血糖监听
[DHBLECommand getBloodSugarMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHBloodSugarModeSetModel *model = data;
    }
}];

```

```
        NSLog(@"getBloodSugarMode OK 开关 %d", model.isOpen);
    }
}];
```

3.2.2.2.6 血压检测设置与获取

间隔血压只可设置60分钟;

配置表属性: `isDataTypeBloodPressure`

方法说明:

```
+(void)setBpMode:(DHBpModeSetModel *)model block:(void(^)(int code, id data))block
```

```
+(void)getBpMode:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
model	DHBpModeSetModel	类	isOpen: true开/false关 interval: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 6. 设置血压监听
DHBpModeSetModel *tModeSetModel = [[DHBpModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBleCommand setBpMode:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setBpMode OK");
    }
}];

//6. 获取血压监听
[DHBleCommand getBpMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHBpModeSetModel *model = data;
        NSLog(@"getBpMode OK 开关 %d", model.isOpen);
    }
}];
```

3.2.2.2.7 体温检测设置与获取

间隔体温可设置30分钟与60分钟;

配置表属性: `isSupportTemperatureMonitoring`

方法说明:

`+(void)setTimedBodyTemperature:(DHHeartRateModeSetModel *)model block:(void(^)(int code, id data))block`

`+(void)getTimedBodyTemperature:(void(^)(int code, id data))block`

参数说明:

参数	类型	说明	值
model	DHHeartRateModeSetModel	类	isOpen: true开/false关 interval: 间隔时间30或60分钟 startHour: 0 固定 startMinute: 0 固定 endHour: 23 固定 endMinute: 59 固定

调用示例:

```
// 7. 设置体温监听
DHHeartRateModeSetModel *tModeSetModel = [[DHHeartRateModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00;
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23;
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60;
[DHBleCommand setTimedBodyTemperature:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){ NSLog(@"setTimedBodyTemperature OK"); }
}];

//7. 获取体温监听
[DHBleCommand getTimedBodyTemperature:^(int code, id _Nonnull data) {
    if (code == 0){
        DHHeartRateModeSetModel *model = data;
        NSLog(@"getTimedBodyTemperature OK 开关 %d 周期 %zd", model.isOpen, model.interval);
    }
}];
```

3.2.2.3 全天检测-同步健康历史数据

同步健康历史数据, 会根据设备支持情况自动同步对应健康数据.

data为数组,为获取到的对应类型的多天数据, 会依次返回对应类型数据, data里不会有多种类型数据.

```
[DHBleCommand startDataSyncing:^(int code, id data){
    NSLog(@"同步完成 %d", code);
} dataBlock:^(int code, int progress, id _Nonnull data) {
    if (code == 0) {
        if ([data isKindOfClass:[NSArray class]]) {
            NSArray *array = data;
            for (id model in array) {
                if ([model isKindOfClass:[DHDailyStepModel class]]) { //计步数据
                    NSLog(@"同步有 计步数据");
                }
            }
        }
    }
}];
```

```

    }
    else if ([model isKindOfClass:[DHDailySleepModel class]]) { //睡眠数据
        NSLog(@"同步有 睡眠数据");
    }
    else if ([model isKindOfClass:[DHDailyHrModel class]]) { //心率数据
        NSLog(@"同步有 心率数据");
    }
    else if ([model isKindOfClass:[DHDailyBoModel class]]) { //血氧数据
        NSLog(@"同步有 血氧数据");
    }
    else if ([model isKindOfClass:[DHDailyHrvModel class]]) { ///HRV数据
        NSLog(@"同步有 HRV数据");
    }
    else if ([model isKindOfClass:[DHDailyPressureModel class]]) { ///压力数据
        NSLog(@"同步有 Stress数据");
    }
    else if ([model isKindOfClass:[DHDailyBloodSugarModel class]]) { ///血糖数据
        NSLog(@"同步有 血糖数据");
    }
    else if ([model isKindOfClass:[DHDailyMuslimCountModel class]]) { ///赞念数据
        NSLog(@"同步有 赞念数据");
    }
    else if ([model isKindOfClass:[DHDailyTempModel class]]) { ///体温数据
        NSLog(@"同步有 体温数据");
    }
    else if ([model isKindOfClass:[DHDailyBpModel class]]) { ///血压数据
        NSLog(@"同步有 血压数据");
    }
}
}
}
}
};

```

3.2.2.4 全天检测-健康数据说明

1. 今天与历史计步数据 DHDailyStepModel

今天与历史计步数据都使用 `DHDailyStepModel`。今天数据通常返回一个当天模型；历史数据可能返回多个日期模型，每个模型代表一天。

数据	返回内容	每日总数
今天计步	当天一个 <code>DHDailyStepModel</code>	使用设备返回的当天总步数、总卡路里和总里程
历史计步	可能包含多个日期的 <code>DHDailyStepModel</code>	对当天历史明细的步数、卡路里和里程分别求和

```

@interface DHDailyStepModel : NSObject

/// 日期时间戳 (秒)
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

/// 里程 (米)
@property (nonatomic, assign) NSInteger distance; //当天的总距离
/// 消耗 (卡路里)

```

```

@property (nonatomic, assign) NSInteger calorie; //当天总卡路里
/// 步数 (步)
@property (nonatomic, assign) NSInteger step; //当天总步数

/// 计步明细间隔, 单位分钟; 未配置时默认60
@property (nonatomic, assign) NSInteger activityDataInterval;

/// 每条包含timestamp、index、step、calorie、distance
/// timestamp为Unix秒; index为当天按当前计步粒度划分后的明细序号
@property (nonatomic, strong) NSMutableArray <NSDictionary *>*items;

@end

```

activityDataInterval 表示 items 的计步明细间隔, 单位为分钟;
 60 表示每小时一条, 10 表示每10分钟一条, 未配置时默认为 60。
 请使用 timestamp 作为明细的准确时间。

2. 睡眠数据 DHDailySleepModel

```

@interface DHDailySleepModel : NSObject

/// 时间戳 (秒)
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

/// 总时长 (分钟)
@property (nonatomic, assign) NSInteger duration;
/// 入睡时间 (时间戳 (秒))
@property (nonatomic, copy) NSString *beginTime;
/// 醒来时间 (时间戳 (秒))
@property (nonatomic, copy) NSString *endTime;

/// 睡眠项 例: @[ @{@"status":@0,@"value":@60},...]
/// status (睡眠类型) value (时长 (分钟))
/// status (0.清醒 1.浅睡 2.深睡, 3. REM)
@property (nonatomic, strong) NSMutableArray <NSDictionary *>*items;

@end

```

3. 心率数据 DHDailyHrModel;

```

@interface DHDailyHrModel : NSObject

/// 时间戳 (秒)
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

/// 心率项 例: @[ @{@"timestamp":@0,@"value":@80},...]
/// timestamp (时间戳 (秒)) value (心率值)
@property (nonatomic, strong) NSMutableArray <NSDictionary *>*items;

@end

```

HRV `DHDailyHrvModel` , 压力 `DHDailyPressureModel` , 血氧 `DHDailyBoModel` , 血糖 `DHDailyBloodSugarModel` 与心率类似可参考对应类

4. Muslim赞念数据 DHDailyMuslimCountModel

```

@interface DHDailyMuslimCountModel : NSObject
/// 时间戳 (秒)
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

@property (nonatomic, assign) NSInteger muslimcount;

/// 赞念项 例: @[ @{@"timestamp":@0,@"value":@80},...]
/// timestamp (时间戳 (秒)) value (每小时赞念值)
@property (nonatomic, strong) NSMutableArray <NSDictionary *>*items; //注意,这里的每小时为累加值赞念

@end

```

5. 体温数据 DHDailyTempModel

```

@interface DHDailyTempModel : NSObject

/// 时间戳 (秒)
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

/// 体温项 例: @[ @{@"timestamp":@0,@"value":@365},...]
/// timestamp (时间戳 (秒)) value (体温值*10, 如365=36.5°C)
@property (nonatomic, strong) NSMutableArray <NSDictionary *>*items;

@end

```

3.2.3 OTA升级

① Note

ota升级文件需从厂家生成取得, 确定无误后再进行测试. 防止升级出错变砖.

方法说明:


```
+(void)ringOtaWithFileData:(NSData *)fileData block:(void(^)(int code, CGFloat progress, id data))block
```

参数说明:

参数	类型	说明
fileData	NSData	固件文件数据
block	Block	进度与结果回调, code=0进行中, progress为进度(0-1)

调用示例:

```
NSString *tFilePath = @""; //bin文件路径,厂家提供
NSData *fileData = [NSData dataWithContentsOfFile:tFilePath];
[DHbleCommand ringOtaWithFileData:fileData block:^(int code, CGFloat progress, id _Nonnull data) {
    NSLog(@"OTA code %d progress %.2f", code, progress);
}];
```

3.2.4 多运动Workout

 **Caution**

支持多运动配置表属性为 `isSupportWorkout3`; 开启多运动后, 设备会进入运动中, APP断开与关闭都不会停止, 只有通过APP或设备主动停止, 所以带多运动功能的设备, 连接后先查询下状态, 确定是否在运动中, 在多运动状态下会影响其它功能使用;

运动时长需超过2分钟, 设备才会保存此次运动数据.

3.2.4.1 获取设备多运动状态

获取设备是否在多运动中; 当前不在运动中时才开启新的运动.

方法说明:

```
+(void)getControlSportWithRing:(void(^)(int code, id data))block
```

参数说明:

WorkoutControlType枚举	类型	说明	值
Workout_Begin	Int	整形	运动开始
Workout_Continue	Int	整形	运动继续
Workout_Pause	Int	整形	运动暂停
Workout_Finish	Int	整形	运动结束

调用示例:

```
[DHbleCommand getControlSportWithRing:^(int code, id _Nonnull data) {
    // @{@"keySportType":@(tSportType), @"keyControlType":@(tControlType)}
    if (code == 0 && [data isKindOfClass:[NSDictionary class]]){
        NSDictionary *tDic = data;
        NSInteger tSportType = [tDic[@"keySportType"] integerValue];
        WorkoutControlType tControlType = [tDic[@"keyControlType"] integerValue];
    }
}];
```

3.2.4.2 控制设备进入多运动

控制设备进入多运动, 启动运动.

运动中数据变化通过接收 BluetoothNotificationRingRuningData 通知获取.

方法说明:

```
+(void)controlSportWithRing:(DHSportControlModel *)model block:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
DHSportControlModel	类		controlType:参考WorkoutControlType sportType: 参考BleActivityMode

运动数据变化通知返回数据说明:

参数	类型	说明	值
ActivityTime	Int	整形	运动持续时间,单位 秒(s);
ActivitySteps	Int	整形	运动中步数
ActivityDistance	Int	整形	运动中产生距离, 单位 米(m);
ActivityCalorie	Int	整形	运动中产生热量, 单位 卡(cal);
ActivityHr	Int	整形	运动中动态心率
ActivityDataType	Int	整形	来源类型, setRingEnterWorkOut也会返回此数据.

调用示例:

```
DHSportControlModel *model = [[DHSportControlModel alloc] init];
model.controlType = Workout_Begin; //开始
model.sportType = tbleActivityMode;
[DHBleCommand controlSportWithRing:model block:^(int code, id _Nonnull data) {
    if (code == 0){
        WorkoutRunningController *runningC = [[WorkoutRunningController alloc]
initWithNibName:@"WorkoutRunningController" bundle:nil];
        runningC.bleActivityMode = tbleActivityMode;
        runningC.controllType = Workout_Begin; //开始
        runningC.modalPresentationStyle = UIModalPresentationOverFullScreen;
        [weakSelf presentViewController:runningC animated:YES completion:^(
        });
    }
}];

[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(ringRuningDataUdpate:)
name:BluetoothNotificationRingRuningData object:nil];
```

```

- (void)ringRuningDataUdpate:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    if (tUserInfo.count > 0){
        NSInteger duration = [tUserInfo[@"ActivityTime"] integerValue];
        NSInteger totalStep = [tUserInfo[@"ActivitySteps"] integerValue];
        NSInteger activityDistance = [tUserInfo[@"ActivityDistance"] integerValue];
        NSInteger activityCalorie = [tUserInfo[@"ActivityCalorie"] integerValue];
        NSInteger activityHr = [tUserInfo[@"ActivityHr"] integerValue];
        NSInteger tActivityDataType = [tUserInfo[@"ActivityDataType"] integerValue];

        NSLog(@"ringRuningDataUdpate tActivityDataType %04X", (UInt32)tActivityDataType);
        NSInteger hours = duration / 3600;
        NSInteger minutes = (duration % 3600) / 60;
        NSInteger seconds = duration % 60;
        self.workoutTimeLb.text = [NSString stringWithFormat:@"Time: %02ld:%02ld:%02ld", (long)hours,
        (long)minutes, (long)seconds];
        self.workoutStepLb.text = [NSString stringWithFormat:@"Steps: %ld", totalStep];
        self.workoutDistanceLb.text = [NSString stringWithFormat:@"Distance: %.2f Km",
        floor(activityDistance/1000.0*100)/100];
        self.workoutCaloriesLb.text = [NSString stringWithFormat:@"Calories: %.1f KCal",
        floor(activityCalorie/1000.0*10)/10];
        self.workoutHeartLb.text = [NSString stringWithFormat:@"HeartRate: %ld bpm", activityHr];
    }
}

```

BleActivityMode 对应名字见示例Demo 字符串里定义:



3.2.4.3 控制开启/关闭设备实时通知运动数据

控制开启/关闭设备实时通知运动数据;

运动中数据变化通过接收 `BluetoothNotificationRingRuningData` 通知获取,有时app关闭与进入后台,可告诉设备停止通知数据.

方法说明:

```

+(void)setRingEnterWorkOut:(UInt8)isEnter block:(void(^)(int code, id data))block

```

参数说明:

参数	类型	说明	值
isEnter	Int	整形	1: 开启通知运动数据; 0: 关闭通知运动数据;

调用示例:

```
//退出运动界面
[DHBleCommand setRingEnterWorkOut:0 block:^(int code, id _Nonnull data) {

}];
```

3.2.4.4 获取多运动数据报告

方法说明:

```
+(void)startRingWorkout3Syncing:(void(^)(int code, id data))block dataBlock:(void(^)(int code, int progress, id data))dataBlock
```

返回数据DHDailySportModel参数说明:

DHDailySportModel类	类型	说明	值
timestamp	NSString		运动开始时间戳
date	NSString		日期yyyyMMdd
viewType	Int		当前运动类型有无步数,步频,配速,距离: 有步频 viewTypeHaveStepFaq: 无步数 viewTypeNoStepNum: 有配速 viewTypeHavePace: 无距离 viewTypeNoDistance:
heartRateItems	数组		当前运动产生的心率列表, 1分钟保存一个;
pacePerKmlItems	数组		每公里配速列表, 单位秒/公里; 如设备不支持则为空
.....			其它属性见类注释

调用示例:

```
[DHBleCommand startRingWorkout3Syncing:^(int code, id _Nonnull data) {
    NSLog(@"startRingWorkout3Syncing 同步完成 code %d", code);
} dataBlock:^(int code, int progress, id _Nonnull data) {
    if (code == 0) {
        if ([data isKindOfClass:[NSArray class]]){
            NSArray *array = data;
            for (id model in array) {
                if ([model isKindOfClass:[DHDailySportModel class]]) {
                    //保存数据库或其它操作

                }
            }
        }

        NSLog(@"startRingWorkout3Syncing data %zd", array.count);
    }
}];
```

5.2.5 传感器原始数据

PPG/ACC/PPG Red/IR传感器原始数据采集与睡眠实时数据;

配置表属性: `isSupportSensorRawPPG` (PPG), `isSupportSensorRawACC` (ACC), `isSupportSensorRawPPGRed` (PPG Red), `isSupportSensorRawIR` (IR), `isSupportSensorRawSleep` (睡眠实时数据);

注意: 睡眠实时数据(sensorType=5)无需手动启动与关闭, 设备支持此功能时会在睡眠过程中自动推送, 通过相同的通知 `BluetoothNotificationHealthRingSenorRawChange` 接收即可.

sensorType 合法组合:

值	含义	说明
1	ACC	仅ACC
2	绿光(PPG Green)	仅绿光
3	绿光 + ACC	绿光与ACC同时输出
4	红光(PPG Red)	仅红光
5	红光 + ACC	红光与ACC同时输出
10	绿光 + 红外(IR)	绿光与红外同时输出
11	绿光 + ACC + 红外	绿光、ACC与红外同时输出
12	红光 + 红外	红光与红外同时输出
13	红光 + ACC + 红外	红光、ACC与红外同时输出

规则: 绿光与红光不能共存; 红外不能单独启动,必须与绿光或红光组合使用.

返回数据格式说明:

字段	说明
sensorType	类型: 1=PPG, 2=ACC, 3=PPG Red, 4=IR, 5=睡眠实时数据
ppgData	PPG数据数组, 每项为int32
accData	ACC数据数组, 每项为{x,y,z} (int16)
ppgRedData	PPG Red数据数组, 每项为int32
irData	IR红外数据数组, 每项为int32
sleepData	type=5时的睡眠数据数组, 每项为{timestamp, mode}; mode: 17=睡眠开始, 34=睡眠结束, 1=深睡, 2=浅睡, 3=清醒, 4=REM

5.2.5.0 PPG定时监测

PPG定时监测设置, 类似心率/HRV定时监测;

配置表属性: `isSupportPPGMonitoring`

方法说明:

```
+(void)setPPGMode:(DHHrvModeSetModel *)model block:(void(^)(int code, id data))block
```

```
+(void)getPPGMode:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
model	DHHrvModeSetModel	类	isOpen: true开/false关 interval: 间隔时间默认30分钟 startHour: 0 固定 startMinute: 0 固定 endHour: 23 固定 endMinute: 59 固定

调用示例:

```
//设置PPG监听
DHHrvModeSetModel *tModeSetModel = [[DHHrvModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00;
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23;
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60;
[DHBleCommand setPPGMode:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){ NSLog(@"setPPGMode OK"); }
}];

//获取PPG监听
[DHBleCommand getPPGMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHHrvModeSetModel *model = data;
        NSLog(@"getPPGMode OK 开关 %d", model.isOpen);
    }
}];
```

5.2.5.1 启动与关闭传感器原始数据

block 回调 code==0 表示启动/关闭成功;
设备也可能主动停止传感器, 通过 BluetoothNotificationHealthRingSenorStopChange 通知.

方法说明:

```
+(void)ringControlSensorRaw:(UInt8)outputType type:(UInt8)sensorType block:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
outputType	UInt8	输出控制类型	1: 开启Sensor输出 2: 关闭Sensor输出
sensorType	UInt8	传感器类型(按位组合)	见上方合法组合表

调用示例:

```

//监听设备主动停止传感器
[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(sensorStopChange:)
name:BluetoothNotificationHealthRingSensorStopChange object:nil];

- (void)sensorStopChange:(NSNotification *)ntf {
    NSLog(@"设备主动停止传感器");
}

//开启PPG+ACC原始数据输出 (sensorType=3)
[DHBleCommand ringControlSensorRaw:1 type:3 block:^(int code, id data) {}];

//关闭PPG+ACC原始数据输出
[DHBleCommand ringControlSensorRaw:2 type:3 block:^(int code, id data) {}];

```

5.2.5.2 数据获取方式

传感器原始数据有两种获取方式, 由设备端决定使用哪种,APP不可选择:

- (1) 实时推送: 启动后设备实时推送数据到APP;
- (2) 历史获取: 设备先采集保存,APP后续主动同步获取;

5.2.5.2.1 实时推送

启动传感器后, 设备实时推送原始数据;

通过 `BluetoothNotificationSensorRawData` 通知返回.

调用示例:

```

[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(sensorRawDataUpdate:)
name:BluetoothNotificationSensorRawData object:nil];

- (void)sensorRawDataUpdate:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    NSInteger tType = [tUserInfo[@"sensorType"] integerValue];
    if (tType == 2) {
        NSLog(@"ACC count=%zd", [tUserInfo[@"accData"] count]);
    } else if (tType == 1) {
        NSLog(@"PPG count=%zd", [tUserInfo[@"ppgData"] count]);
    } else if (tType == 3) {
        NSLog(@"PPG Red count=%zd", [tUserInfo[@"ppgRedData"] count]);
    } else if (tType == 4) {
        NSLog(@"IR count=%zd", [tUserInfo[@"irData"] count]);
    } else if (tType == 5) {
        NSArray *sleepData = tUserInfo[@"sleepData"];
        NSLog(@"Sleep count=%zd", sleepData.count);
    }
}

```

5.2.5.2.2 历史获取

获取设备保存的传感器历史原始数据, 类似多运动数据同步方式;

数据通过 `dataBlock` 回调返回, `block` 的 `code==0` 表示同步完成.

方法说明:

```
+(void)ringGetHistorySensorRaw:(void(^)(int code, id data))block dataBlock:(void(^)(int code, int progress, id data))dataBlock
```

dataBlock 返回 NSArray, 每个元素包含以下字段:

字段	说明
sensorType	传感器类型(1:PPG 2:ACC 3:PPG Red 4:IR)
sequence	序号
count	数据个数
ppgData	PPG数据数组(sensorType==1时)
accData	ACC数据数组(sensorType==2时), 每项含x,y,z
ppgRedData	PPG Red数据数组(sensorType==3时)
irData	IR数据数组(sensorType==4时)

dataBlock 只回调一次, 传入完整结果数组。progress 固定为100。

调用示例:

```
[DHBleCommand ringGetHistorySensorRaw:^(int code, id _Nonnull data) {
    NSLog(@"Sensor History sync finished code %d", code);
} dataBlock:^(int code, int progress, id _Nonnull data) {
    if (code == 0 && data) {
        NSArray *resultArray = data;
        for (NSDictionary *info in resultArray) {
            NSLog(@"sensorType=%@ seq=%@ count=%@", info[@"sensorType"], info[@"sequence"],
info[@"count"]);
        }
    }
}];
```

SDK修订记录

V2.0.0_20260724 (2026.07.24)

- 添加计步明细间隔支持

V2.0.0_20260706 (2026.07.06)

- 添加外部 `CBCentralManager` 接入方式(3.1.8), 支持客户统一搜索设备后由 SDK 使用同一个 Central 发起连接。

V2.0.0_20260616 (2026.06.16)

- 添加计数提醒间隔设置功能(3.2.1.25)
- 功能配置表添加 `isSupportCountReminder`
- 睡眠数据items新增 `isTemporary` 字段(1:临时数据 0:正式数据)

V2.0.0_20260610 (2026.06.10)

- 添加定时体温监测功能(3.2.2.2.7)
- 功能配置表添加 `isDataTypeTemperature`

- 添加跌落提醒设置功能(3.2.1.24)
- 功能配置表添加 `isSupportFallDetect`

V2.0.0_20260522 (2026.05.22)

- 添加心率校正功能(3.2.1.23)
- `DHPeripheralModel` 新增设备型号 `deviceModel` 属性

V2.0.0_20260507 (2026.05.07)

- 添加睡眠实时数据(`sensorType=5`)支持

V2.0.0_20260505 (2026.05.05)

- 添加震动间隔时长设置(3.2.1.22)

V2.0.0_20260429 (2026.04.29)

- 添加PPG定时监测功能(5.2.5.0)

V2.0.0_20260428 (2026.04.28)

- OTA改成 `ringOtaWithFileData` 接口

V2.0.0_20260414 (2026.04.14)

- 健康实时数据返回格式优化

V2.0.0_20260408 (2026.04.08)

- 添加传感器原始数据历史获取功能(5.2.5.2.2)
- 添加闹钟震动时长设置(3.2.1.20)
- 修改设置睡眠模式、功能表用错问题
- 添加触摸事件通知(3.2.1.21)

V2.0.0_20260327 (2026.03.27)

- 多运动报告数据添加每公里配速(`pacePerKmItems`)字段

V2.0.0_20260314 (2026.03.14)

- 传感器原始数据添加PPG Red(`type=3`)和IR红外(`type=4`)类型
- 功能配置表添加 `isSupportSensorRawPPGRed` 和 `isSupportSensorRawIR`

V2.0.0_20260309 (2026.03.09)

- 处理读取亮屏等级Bug
- PPG/ACC传感器原始数据加 `type=0` 输出

V2.0.0_20260303 (2026.03.03)

- 添加Muslim计数清零方式设置与获取功能

V2.0.0_20260302 (2026.03.02)

- 加血压监测功能
- 加Muslim时间显示模式
- 加PPG/ACC传感器原始数据接口

V2.0.0_20260225 (2026.02.25)

- 添加[设置时间格式12/24小时制](#)

V2.0.0_20260208 (2026.02.08)

- OTA升级加超时处理

V2.0.0_20260118 (2026.01.18)

- 加SDK版本号接口
- 加功能配置表
- 加震动、睡眠模式、消息推送、报警等设置新功能指令
- 加多运动功能指令

1.0.6 (2026.01.15)

- 添加赞念客户新需求

1.0.5 (2026.01.05)

- 加设置亮屏时长时关闭睡眠模式功能

1.0.4 (2025.08.21)

- 赞念可设置测试值
- 启动单次赞念也实时同步数据

1.0.3 (2025.08.12)

- 添加Muslim定制功能
- 戒指产品去掉第三方Framework

1.0.2 (2025.07.23)

- 添加Muslim定制产品相关功能

1.0.1 (2025.04.08)

- 添加戒指产品功能

1.0.0 (2024.12.01)

- 添加电子烟新功能

联系方式 / 技术支持

- 技术支持邮箱 developer@dhhouse88.com